MASTERING .NET PROGRAMMING

UNIT II — CLASS LIBRARIES, ASSEMBLIES & C# FUNDAMENTALS

Beginner-friendly teaching notes • Definitions • Why we need it • Step-by-step concepts • Examples • C# code • Practice

1. What is a Class Library?

Definition: A class library is a collection of reusable classes, interfaces, structures and other types that provide ready-made functionality to programs.

A beginner can think of a class library as a **toolbox**. Instead of building every tool yourself, you select a suitable class or method from the toolbox and use it.

The .NET Framework provides a large set of libraries for common tasks such as input/output, collections, text processing, dates, networking and many other operations.

```
using System;
using System.Collections.Generic;

List<int> marks = new List<int>();
marks.Add(80);
marks.Add(90);

Console.WriteLine(marks.Count);
```

Explanation: List<int> is a reusable collection class. The programmer uses Add() and Count instead of implementing an entire dynamic list from scratch.

> **Why do we need class libraries?**
> They save time, reduce duplicate code, provide tested/reusable functionality and give programmers a standard programming model.

2. .NET Class Library and Namespaces

The framework organizes many types into namespaces. A namespace helps us find related functionality and prevents naming conflicts.

```
using System;
using System.IO;

Console.WriteLine("Hello");
File.WriteAllText("student.txt", "Aman");
```

System contains many basic types and services. System.IO contains file and stream related types.

Namespace	Examples of functionality
System	Console, String, Math, Object and many fundamental types
System.IO	Files, directories and streams
System.Collections.Generic	Generic collections such as List<T> and Dictionary<TKey,TValue>

3. Assemblies in .NET

Definition: An assembly is a compiled unit of .NET code. It commonly takes the form of an EXE or DLL and contains information required by the runtime.

Assemblies are important because they provide a unit for **deployment, versioning, reuse and identity**.

Type	Typical purpose
EXE	An executable application that can be launched.
DLL	A reusable library/component that can be referenced by applications.

Real-life analogy
Think of an assembly as a packaged box. The box contains compiled code and information describing the package. The application or runtime can use the package when required.

4. What is a DLL?

DLL stands for **Dynamic Link Library**. In .NET, a DLL is commonly used to store reusable classes and functionality that another application can reference.

```
// Example library class
public class Calculator
{
public int Add(int a, int b)
{
return a + b;
}
}
```

The library could be compiled into a DLL. Another application could reference the DLL and create a Calculator object.

Why use DLLs?
If several applications need the same business logic, placing that logic in a reusable library can reduce duplication and make maintenance easier.

5. What is an Assembly Manifest?

Definition: The manifest is information stored in an assembly that describes the assembly's identity and dependencies.

• Assembly name.

• Version information.

• Culture information where applicable.

• Referenced assemblies.

• Information about modules contained in the assembly.

The manifest helps the .NET runtime understand what an assembly is and what other components it depends on.

Simple idea
The manifest is like the label on a packaged product: it tells you the product's identity, version and important dependency information.

6. Assembly Identity and Version

Version information becomes important when applications are maintained over time. A library may be updated while applications continue to reference it.

Concept	Meaning
Name	Identifies the assembly.
Version	Identifies the release/version of the assembly.
Culture	Can identify culture-specific resources.
Dependencies	Identify assemblies needed by the current assembly.

7. Metadata

Definition: Metadata is **data about the program**. It describes information such as classes, methods, fields, properties, parameter types, access modifiers and inheritance relationships.

```
class Student
{
public int Age;

public void Study()
{
}
}
```

Conceptually, the compiled program contains information that allows .NET tools and runtime services to understand that Student is a class, Age is an integer field and Study is a method.

> **Real-life analogy**
> A student record contains data about a student. Metadata similarly contains information about the structure of the program itself.

8. Why is Metadata Important?

• Helps the runtime understand compiled types.

• Supports reflection and tools that inspect program structure.

• Supports type information needed by the runtime.

• Makes compiled components self-describing to an important extent.

Students should remember: **source code tells the programmer what the program does; metadata describes the structure of the compiled program.**

9. Attributes in C#

Definition: An attribute is a declarative annotation that attaches additional information to a program element such as a class, method or property.

Attributes are written inside square brackets before the program element.

```
[Obsolete("Use CalculateNew() instead")]
public int CalculateOld()
{
return 10;
}
```

The Obsolete attribute tells the compiler/tools that CalculateOld() should no longer normally be used and provides a replacement message.

Concept	Meaning
Metadata	Information describing the structure of compiled code.
Attribute	A declarative annotation added to a program element.

10. Introduction to C#

Definition: C# is a strongly typed, object-oriented programming language designed for the .NET ecosystem.

C# supports classes, objects, inheritance, polymorphism, interfaces, exception handling, generics, delegates, events and many other features.

```
using System;

class Program
{
static void Main()
{
```

```
Console.WriteLine("Welcome to C#");
}
}
```

This is a basic console application. Main() is the entry point in this traditional form of program.

11. Important Characteristics of C#

• **Object-oriented:** Supports classes and objects.

• **Strongly typed:** Variables have defined types and the compiler checks type usage.

• **Type-safe:** The language and runtime provide mechanisms that help prevent invalid type operations.

• **Reusable:** Classes and libraries can be reused.

• **Exception handling:** Supports structured error handling.

• **Interoperability:** Works with the .NET runtime and its common type system.

12. Data Types in C#

Definition: A data type tells the compiler what kind of value a variable can store and what operations are valid for that value.

Category	Examples	Simple meaning
Integral	int, long, short, byte	Whole numbers
Floating point	float, double	Numbers with fractional parts
Boolean	bool	true or false
Character	char	One character
String	string	Sequence of characters
Object	object	Base reference type for all .NET types

```
int age = 20;
double price = 99.50;
bool passed = true;
char grade = 'A';
string name = "Aman";
```

13. Value Types

Definition: A value type directly contains its value. Common examples include int, double, bool, char, structures and enumerations.

```
int a = 10;
int b = a;

b = 50;

Console.WriteLine(a); // 10
Console.WriteLine(b); // 50
```

Why is a still 10? When b = a executes, b receives a copy of the value. Changing b does not change a.

> **Beginner analogy**
> Photocopying a paper: you now have two separate copies. Changing one copy does not change the other.

14. Reference Types

Definition: A reference type variable stores a reference to an object rather than directly containing the complete object data.

```
class Student
{
```

```
public string Name;
}

Student s1 = new Student();
s1.Name = "Aman";

Student s2 = s1;
s2.Name = "Ravi";

Console.WriteLine(s1.Name); // Ravi
```

Both s1 and s2 refer to the same Student object. Therefore, changing the object's Name through s2 is visible through s1.

Value Type	Reference Type
Stores value directly	Stores a reference to an object
Example: int	Example: class Student
Assignment copies the value	Assignment copies the reference

15. Default Values

A default value is the value obtained when a variable/type is initialized with the default value mechanism.

Type	Typical default
int	0
double	0.0
bool	false
char	\0
Reference types	null

A useful distinction for beginners: fields receive default values automatically, while local variables must be definitely assigned before they are read.

16. Constants

Definition: A constant is a value that is fixed and cannot be reassigned after declaration.

```
const double GST = 0.18;

double price = 1000;
double tax = price * GST;

Console.WriteLine(tax);
```

Constants make code readable and prevent accidental changes to values that should remain fixed.

> **Common examples**
> Mathematical constants, fixed tax rates, application limits or other values that should not be changed during execution.

17. Variables

Definition: A variable is a named storage location whose value can change during program execution.

```
int age = 20;
age = 21;

Console.WriteLine(age);
```

Here age first contains 20 and later contains 21.

Term	Meaning
Declaration	Telling the compiler the variable's type/name.
Initialization	Giving a variable its first value.
Assignment	Giving/replacing the value stored in a variable.

```
int marks; // declaration
marks = 80; // assignment

int age = 20; // declaration + initialization
```

18. Scope of Variables

Scope means the region of a program in which a variable can be accessed.

Scope	Example
Local variable	Declared inside a method/block; available only within its accessible region.
Instance field	Belongs to an object and is accessed through that object/member rules.
Static field	Belongs to the type and is shared by the type's instances.

```
class Counter
{
int value = 0; // instance field
static int total = 0; // static field

void Run()
{
int x = 10; // local variable
Console.WriteLine(x);
}
}
```

A local variable x cannot normally be accessed outside the method/block in which it is declared.

19. Boxing

Definition: Boxing is the conversion of a value type into an object reference.

```
int marks = 90;

object data = marks; // boxing
```

The value 90 is placed into an object representation so that it can be treated as an object.

> **Think of boxing**
> A value is placed inside a container so that code expecting an object can carry it.

20. Unboxing

Definition: Unboxing is the explicit extraction of a value type from an object containing a compatible boxed value.

```
object data = 90;

int marks = (int)data; // unboxing

Console.WriteLine(marks);
```

The cast is explicit because the runtime needs to know which value type should be extracted.

> **Important**
> Unboxing must use a compatible value type. An incompatible cast can cause an InvalidCastException.

21. Boxing and Unboxing Together

```
int number = 25;

object boxed = number; // Boxing

int result = (int)boxed; // Unboxing

Console.WriteLine(result);
```

Step 1: number is an int. Step 2: assigning it to object boxes the value. Step 3: casting the object back to int unboxes it.

In modern C# programming, generic collections such as List<int> often help avoid unnecessary boxing when storing value types.

22. Complete Beginner Example — Student Information

```
using System;

class Student
{
public string Name;
public int Age;
public double Marks;

public void Display()
{
Console.WriteLine("Name: " + Name);
Console.WriteLine("Age: " + Age);
Console.WriteLine("Marks: " + Marks);
}
}

class Program
{
static void Main()
{
Student s = new Student();

s.Name = "Aman";
s.Age = 20;
s.Marks = 82.5;

s.Display();
}
}
```

What concepts are demonstrated? C# class, reference type, object creation, fields, string/int/double data types, variable assignment and method invocation.

23. Common Beginner Confusions

• **Assembly vs class:** A class is a type definition; an assembly is a compiled package containing code and metadata.

• **Metadata vs attribute:** Metadata describes compiled program structure; an attribute is a declarative annotation added to program elements.

• **Value vs reference type:** Do not simply say 'value is stored in stack and reference in heap' as a universal rule. The key conceptual distinction is value semantics versus references to objects.

• **Boxing vs casting:** Boxing specifically converts a value type to object; unboxing extracts the compatible value type from that object.

• **Constant vs variable:** A constant cannot be reassigned; a normal variable can change.

24. Unit II Quick Revision

Question	One-line answer
What is a class library?	A reusable collection of .NET types and functionality.
What is an assembly?	A compiled unit such as an EXE or DLL containing code and metadata.

Question	One-line answer
What is a manifest?	Assembly information describing identity, version and dependencies.
What is metadata?	Information describing the structure of compiled code.
What is an attribute?	A declarative annotation attached to a program element.
What is a value type?	A type with value semantics, such as int.
What is a reference type?	A type whose variable refers to an object.
What is boxing?	Converting a value type to object.
What is unboxing?	Explicitly extracting a compatible value type from an object.

25. Practice Questions

• What is a class library? Why do we need class libraries?

• Explain EXE and DLL assemblies with examples.

• What is an assembly manifest? What information does it contain?

• Define metadata and explain why it is useful.

• What are attributes? Explain the Obsolete attribute with code.

• Explain the important characteristics of C#.

• Define value types and reference types with programs.

• Explain variables, constants and scope.

• Explain boxing and unboxing with a complete example.

• Write a C# program using different data types and explain every variable.

26. Teaching Checklist

• Start with the toolbox analogy before introducing class libraries.

• Show students the difference between an EXE and DLL using a simple application/library example.

• Explain manifest and metadata conceptually before showing technical terminology.

• Demonstrate value-type copying with int before teaching reference-type behavior with a class.

• Run the boxing/unboxing program and explain exactly what changes from int to object and back.

• Finish with the Student example and ask students to identify the class, object, fields, data types and method.